

PDM Planning (15 weeks)

Academic period: 07/09/2026 to 19/12/2026

Publication of the statements: 09/14/2026

Deadline for submission of assignments: 12/12/2026

Discussions held from: 12/14/2026

Week 1 (07/09/2026)

Topic: Presentation

- Program
- Objectives and framework
- Teaching methodology and evaluation rules
- Preparing the work environment

For reference:

- [Download Android Studio & App Tools](#)
- [Android API Levels](#)
- [SDK Platform release notes | Android Developers](#)

Topic: Anatomy of an Android application

- Component Activity: introduction
 - Android component for hosting the UI
- Android project composition

Demos:

- Demo *Hello Android*, using the wizard

Topic: Building the UI with Jetpack Compose - TDS revisions

- Jetpack Compose
 - @Composable functions: State → @Composable → UI
 - Stateless @Composables
 - Elementary composables: Text and Button
 - Layouts: Row, Column and Box
 - UI event handling
- Design considerations
 - Separation between domain and UI

- The value of immutability
- Automated testing on Android
 - Automated tests for the domain model
 - Automated UI Testing (Introduction)

Demos:

- Demo *8-Puzzle*, with only one Activity and no view-model
 - Implement the domain model
 - Implement `TileView` and `PuzzleView` (i.e. Stateless Composables)

For reference:

- [Thinking in Compose | Jetpack Compose | Android Developers](#)
- [Compose layout basics - Jetpack](#)
- [Testing your Compose layout | Jetpack Compose | Android Developers](#)
- [Preview your UI with composable previews | Jetpack Compose | Android Developers](#)
- [Immutability we can afford. A look at modern programming languages... | by Roman Elizarov](#)
-  [Keynote The Value of Values - Rich Hickey](#)
- [From Primitive Obsession to Domain Modelling](#)
-  [Prodigi - Curso de PDM - M05 - Compose \(Introdução\)](#)
-  [Prodigi - Curso de PDM - M06 - Compose Layouts](#)

Week 2 (September 14, 2026)

Topic: Building the UI with Jetpack Compose - TDS and PC reviews

- Jetpack Compose
 - Stateful `@Composables`
 - Presentation state management: `remember` and `mutableStateOf`
 - State hoisting
 - Life cycle of Composables and automatic recomposition
- Design considerations
 - Implementing the screen as a state machine.
- Jetpack Compose concurrency model
 - Kotlin's concurrency model, revisited.
 - Elementos: `Threads`, `Coroutines`, `Dispatchers` e `Scopes`
 - Main/UI thread, message queue, and sequential callback execution.
 - `LaunchedEffect` and `rememberCoroutineScope`

Demos:

- Demo *8-Puzzle*, with only one Activity and no view-model
 - Implement the `viewsIdleView`, `SolvingView` and `SolvedView`
 - Implement `PlayScreen` (composable root, screen state machine host)

For reference:

- [State and Jetpack Compose | Android Developers](#)
-  Prodigy - Curso de PDM - M07 - Compose (Estado)

Week 3 (September 21, 2026)

Her: Component Activity

- Life cycle
 - Elementary: onCreate, onStart, onStop and onDestroy
 - Behavior during reconfiguration
 - Implications of reconfigurations in presentation state management

Topic: Building the UI with Jetpack Compose

- Jetpack Compose:
 - Stateful @Composables
 - Presentation status management: remember and mutableStateOf
 - State hoisting
 - Preservation of presentation state with rememberSaveable
- Design considerations
 - Presentation status or application status
- Architecting the UI
 - Support for multiple orientations: LocalConfiguration.current.orientation
- Application resources, introduction
 - Multilingual text
 - Images

Topic: The MVVM pattern in Android

- ViewModel
 - Purpose and applicability
 - Life cycle and relationship with the Activity
 - ViewModelStore and ViewModelStoreOwner
- Design considerations
 - ViewModel while
 - Gateway to the domain model
 - Local application state container to the screen
 - Host of the execution of *use case*
 - Screen as a visual materialization of the state contained in the *view model*

Demos:

- Demo *8-Puzzle*, with only one Activity and with a viewmodel

- The viewmodel hosts the screen's state machine and exposes the current state through a MutableState property.

For reference:

- [Architecting your Compose UI | Jetpack Compose | Android Developers](#)
- [Guide to application architecture](#)
- [ViewModel overview | Android Developers](#)
 - [ViewModel lifecycle](#)
 - [ViewModel Scoping APIs | App architecture | Android Developers](#)

Other links:

- [Compose state explained Series' Articles - DEV Community](#)
- [Support different screen sizes | Android Developers](#)
- [Jetpack Compose – Material Design 3](#)
- [Material Design 3 in Compose](#)
- [Jetpack Compose Material 3 Compose components](#)
- [▶ Prodigy - Curso de PDM - M04 - Activity](#)
- [▶ Prodigy - Curso de PDM - M09 - ViewModel](#)

Week 4 (September 28, 2026)

Topic: Building the UI with Jetpack Compose - navigating between screens

- Component Activity (continuation)
 - Life cycle: behavior in navigating between activities
- UX: navigation between activities
 - *User task* and *back stack*
 - Intents: implicit and explicit
- Jetpack Compose:
 - Composition Local
 - Objectives and applicability
 - Examples of use: LocalConfiguration, MaterialTheme, LocalViewModelStoreOwner, LocalLifecycleOwner
- Automated testing on Android
 - Automated tests for navigating between activities

Demos:

- Demo *8-Puzzle with* multiple activities
 - Implement StartActivity and AboutActivity

For reference:

- [Tasks and the back stack | Android Developers](#)

- [Intents and intent filters | Android Developers](#)
 - [Sending the user to another app | Android Developers](#)
 - [Common intents | Android Developers](#)
- [Locally scoped data with CompositionLocal | Jetpack Compose | Android Developers](#)

Week 5 (October 5, 2026)

Topic: Monitoring of practical work

- Students will need to record a video of up to 7 minutes presenting their application.
- The presentation includes the indication
 - of the features already implemented
 - of the main decisions taken
 - of the strategy for verifying correctness
 - planning the execution of the missing tasks

Week 6 (12/10/2026)

Topic: Android Concurrency Model

- Implementation of change notification methods in the lifecycle.
- Execution in *handlers* of events
- Kotlin Concurrency Model (revisions)
 - *Thread vs coroutine*
 - *Suspending functions, coroutines, scopes and dispatchers*
 - *Flows: flow builder e StateFlow*
- ViewModel, revisited
 - ViewModel as the host of the execution: `viewModelScope`
 - ViewModel as a host for UI state flows
- Updating the UI from state flows: `StateFlow.collectAsStateWithLifecycle()`
- Reactive architecture for Android applications (and more): introduction

Demos:

- **Demo 8-Puzzle**
 - Implement automatic resolution (with Learning Real-Time A* - LRTA*), an incremental resolution algorithm, to better justify the use of streams.
 - Add Solving state to the screen's state machine.`PlayScreen`
 - Use reactive architecture from the domain to the UI.

For reference:

- [Asynchronous Flow | Kotlin Documentation](#)
- [Kotlin flows on Android](#)
 - [StateFlow and SharedFlow | Kotlin | Android Developers](#)

- [State and Jetpack Compose | Android Developers](#)
- [ViewModel overview | Android Developers](#)
 - [Use Kotlin coroutines with lifecycle-aware components | Android Developers](#)
- [Testing Kotlin coroutines on Android](#)
- [Testing Kotlin flows on Android](#)
- [📺 Prodigy - Curso de PDM - M08 - Modelo de Concorrência](#)

Week 7 (10/19/2026)

Topic: State management (continued)

- Running on Android
 - Work *user facing* and *non-user facing*
 - Automated resource management in Android and its consequences for the programming model.
 - Activity and ViewModel lifecycle, revisited
 - *User Task and back stack*, revisited (navigation between activities with information transfer)

Topic: Dependency resolution

- Injection of dependencies: motivation
- Class Application:
 - Motivation and life cycle
 - Used for dependency resolution (as a *Service Locator*)

Demos:

- Demo *8-Puzzle*
 - What happens when the application goes into the background?
 - What happens when the application process is finished?
 - Which state is automatically preserved?
- Demo *8-Puzzle*
 - *Refactoring* for a Single Activity architecture, using Compose Navigation 3
 - Which state is automatically preserved?
- Demo *MyGameVault* (application for managing a video game collection and its gameplay metrics)
 - Implement the screen to add games to the collection using *fake service* and realistic contract (suspending)
 - Implement manual dependency resolution using *Application*

For reference:

- [Processes and app lifecycle | App architecture | Android Developers](#)
- [Parcelables and bundles | App architecture | Android Developers](#)
- [Navigation 3 | App architecture | Android Developers](#)

- [Application | Android Developers](#)
- [Manual dependency injection | Android Developers](#)

Week 8 (October 26, 2026)

Topic: Communication with HTTP APIs

- The Android device as an HTTP client
 - Motivation and consequences of distribution
 - Required permissions
- HTTP communication with Ktor
 - Programming model
 - Asynchronous order fulfillment
- JSON seriation and deserialization with Ktor and Kotlin Serialization
- Design considerations:
 - Regarding the use of proprietary or third-party APIs.
 - DTOs, Domain Objects, and conversion between the two (for third-party APIs)
 - DTOs as Domain Objects (with their own APIs)
 - Regarding the distribution of secrets
 - BYOK, with keys entered by the user and saved in preferences.
 - Backend Proxy, accessible by presenting user credentials.
 - Remote Config with Firebase (proxy backend variant with hosting)
 - Distributed with the application and injected into the build via local.properties
 - Regarding errors (failure model)
 - Consequences of distribution
 - Consequence of using third-party APIs (i.e.(backward compatibility issues and unavailability)
 - About observability

Demos:

- Demo *MyGameVault*
 - Implement a service that makes HTTP requests to the IGDB API ([IGDB API docs](#))
 - Implement error handling.
 - Add observability via logging (without Firebase Crashlytics yet)

For reference:

- [Connect to the network](#)
- [Who is the client?](#)
 - [Ktor Engines](#)
 - [Content Negotiation](#)

- [Kotlin Serialization](#)

Week 9 (02/11/2026)

Topic: Persistent storage on the device

- Storage with DataStore
 - General characterization
 - Description of the data model (i.e., Key-Value pairs)
 - Applicability (low-volume unstructured data)
- Relational database storage via Room.
 - General characterization
 - Applicability

Demos:

- Demo *MyGameVault* Implement MyCollection screen
 - Start by defining the new domain elements (CollectionEntry and abstraction CollectionRepository)
 - Implement the in-memory repository (for simplified development).
 - Implement the repository using Room.

For reference:

- [App Architecture: Data Layer | DataStore | Android Developers](#)
- [Save data in a local database using Room | App data and files | Android Developers](#)

Week 10 (09/11/2026)

Topic: Monitoring of practical work

- Students will need to record a video of up to 7 minutes presenting their application.
- The presentation includes the indication
 - of the features already implemented
 - of the main decisions taken
 - of the strategy for verifying correctness
 - planning the execution of the missing tasks

Week 11 (11/16/2026)

Topic: Execution of background tasks

- Computational resource management in Android
 - Android app lifecycle
 - Process classification: *foreground*, *visible*, *service*, and *cached*
- Work Manager API
 - Motivation and applicability

- Work scheduling
- Competition model of `CoroutineWorker`

Demos:

- Demo *MyGameVault* Implement gameplay time tracking functionality.

For reference:

- [Processes and app lifecycle | App architecture | Android Developers](#)
- [Background tasks overview | Background work | Android Developers](#)
- [Task scheduling | Background work | Android Developers](#)
- [Threading in `CoroutineWorker` | Background work | Android Developers](#)

Week 12 (November 23, 2026)

Topic: Task execution background (*continued*)

- Foreground Services
 - Motivation and applicability
 - Competition model
- Notifications
 - Motivation and applicability

Demos:

- Demo *MyGameVault* Refine the game time tracking feature so that a timer notification is maintained.

For reference:

- [Foreground services overview | Background work | Android Developers](#)
- [About notifications | Jetpack Compose | Android Developers](#)

Week 13 (November 30, 2026)

Topic: Presentation of the remaining elements of the Android programming model

Topic: Consolidation of the My Game Vault demo

Week 14 (07/12/2026)

Topic: Monitoring of practical work

- Students should complete the development activities.
- Students will need to record a video of up to 7 minutes presenting their application.
- The presentation includes
 - a demonstration of the application's functionalities
 - A discussion of the main decisions taken.

Week 15 (14/12/2026)

Topic: Discussions